QueryAPIGate HOW-TO GUIDE 06 · QUERIES

Use bound parameters safely

Current as of QueryAPIGate **v0.49.0** · PDF made 4 November 2026

Latest version: anantharajuc.github.io/QueryAPIGate-website/Guide.dc.html?g=06-use-bound-parameters-safely

Time: 10 minutes. **You'll end up with:** a saved query whose parameters are typed, validated, and documented - rejecting bad input before it ever reaches your database, with every rule enforced server-side, not left to whoever's calling it.

`:name` vs `{name}` - use `:name` unless you have a real reason not to

```sql
SELECT * FROM products WHERE category = :category   -- bound parameter (preferred)
SELECT * FROM products ORDER BY {sort_col}           -- text placeholder (legacy)
```

`:name` is a real bound parameter - the value is sent to the database driver separately from the SQL text (the same mechanism every language's "parameterized query" refers to). It has no SQL-injection surface by construction: a value can contain literally anything - quotes, semicolons, `DROP TABLE`, doesn't matter - because it's never parsed as SQL syntax at all.

`{name}` is substituted as text, before the query runs - because of that, its value is restricted to numbers, booleans, or a string made only of letters, digits, whitespace and `. , : @ % + / -` (anything else, including a `--` comment sequence, is rejected outright). Use it only for what `:name` structurally can't express - an identifier (`ORDER BY {sort_col}`, a column or table name - never data), since a bound parameter can only ever stand in for a *value*, never for SQL syntax itself.

One real, verified difference worth knowing: `{name}` text placeholders only work in a **saved** query - resolved when `GET /q/<name>` runs it. Ad-hoc SQL through `POST /execute_sql` only supports bound `:name` parameters; a `{name}` placeholder there is sent to the database literally and fails as a syntax error. If you need a dynamic identifier ad hoc, build it into a saved query instead.

```shell
# Works - :category is a real bound parameter, ad hoc or saved
curl -X POST http://127.0.0.1:5000/execute_sql -H 'X-API-Key: demo-key' -H 'Content-Type:
application/json' \
  -d '{"sql": "SELECT * FROM products WHERE category = :category", "connection_name": "shop", "params":
{"category": "tools"}}'

# Fails ad hoc - {col} is never substituted outside a saved query
curl -X POST http://127.0.0.1:5000/execute_sql -H 'X-API-Key: demo-key' -H 'Content-Type:
application/json' \
  -d '{"sql": "SELECT * FROM products ORDER BY {col}", "connection_name": "shop", "params": {"col":
"price"}}'
# {"error": "An error occurred while executing the SQL query", "detail": "unrecognized token: \"{\""}
```

Declaring rules with `parameters`

Add a `parameters` object when saving a query - each key is a parameter name used somewhere in the SQL, each value either a bare type string or a full rules object:

shell

```
curl -X POST http://127.0.0.1:5000/api/v1/queries -H 'X-API-Key: demo-key' -H 'Content-Type:
application/json' -d '{
  "name": "products_search",
  "sql": "SELECT id, name, category, price FROM products WHERE (:name IS NULL OR name LIKE :name) AND
category = :category AND price BETWEEN :min_price AND :max_price ORDER BY price",
  "parameters": {
    "name": {"type": "str", "required": false, "min_length": 2, "max_length": 30, "pattern": "[A-Za-z
]+", "description": "Partial name match"},
    "category": {"type": "str", "enum": ["tools", "electronics", "kitchen"], "default": "electronics"},
    "min_price": {"type": "float", "min": 0, "default": 0},
    "max_price": {"type": "float", "max": 1000, "default": 1000}
  },
  "connection_name": "shop", "author": "you", "description": "Search products",
  "publish": true
}'
```

Every rule, exactly as enforced:

RULE	APPLIES TO	MEANING
`type`	any	`int`, `float`, `str` or `bool` (`integer` / `number` / `string` / `boolean` also accepted). Query-string text is converted to this type; a JSON body's value must already have the right type. An undeclared type accepts any single value.
`required`	any	Defaults to `true`, or to `false` automatically once a `default` is given. An optional parameter with no value and no default binds as `NULL` - which is exactly why the query above writes `(:name IS NULL OR name LIKE :name)`, not just `name LIKE :name`.
`default`	any	Used whenever the request supplies nothing. Must itself satisfy every other rule. Can't be combined with `"required": true`.
`enum`	any	The value must be exactly one of these.
`min`, `max`	numbers	Inclusive bounds.
`min_length`, `max_length`	text	Length bounds, in characters.
`pattern`	text	A regular expression the *whole* value must match (at most 500 characters).
`description`	any	Shown in `/docs` (the OpenAPI reference) - documents the parameter for whoever calls this API, not just you.

Calling it, and what a validation failure actually looks like

```shell
curl 'http://127.0.0.1:5000/q/products_search' -H 'X-API-Key: demo-key'
# defaults apply: category=electronics, min_price=0, max_price=1000, name=NULL
```

```shell
curl 'http://127.0.0.1:5000/q/products_search?category=tools&min_price=5' -H 'X-API-Key: demo-key'
# [{"id": 1, "name": "Widget", "category": "tools", "price": 9.99}]
```

Every problem is reported **at once**, in one 400 response, before the query ever touches the database:

```shell
curl 'http://127.0.0.1:5000/q/products_search?category=furniture&min_price=bogus' -H 'X-API-Key: demo-key'
```

```json
{
  "error": "Invalid parameters: category must be one of: tools, electronics, kitchen; min_price must be a number",
  "code": "param_invalid",
  "errors": {"category": "must be one of: tools, electronics, kitchen", "min_price": "must be a number"},
  "request_id": "945fb02c9970"
}
```

A request rejected this way is never recorded in the query's `execution_history` - a bad request from a misbehaving client doesn't pollute the run history you'd actually want to look at later.

A parameter your SQL uses but you never declared

Still works - it's just required, with no validation beyond "a value was supplied," and passed straight through:

```shell
curl 'http://127.0.0.1:5000/q/by_category_undeclared' -H 'X-API-Key: demo-key'
# {"error": "No value provided for parameter(s): category"}
curl 'http://127.0.0.1:5000/q/by_category_undeclared?category=tools' -H 'X-API-Key: demo-key'
# [{"id": 1, "name": "Widget", "category": "tools", "price": 9.99}]
```

Declaring `parameters` is opt-in extra safety and documentation, not a requirement for a parameter to work at all - but an undeclared one gets none of `enum` / `min` / `max` / `pattern`'s protection, and won't show up with a description in `/docs` either.

Your own `LIMIT` is respected

Every response from QueryAPIGate is paginated (`?page` / `?page_size`), and paging happens *within* your query's own window: a "top 3" query (`... ORDER BY price DESC LIMIT 3`) returns 3 rows, never a whole page, and `LIMIT 25` pages through those 25. Use `LIMIT` for what the query means ("the top N"), and `?page_size=` (with a `max` rule on a

parameter if you want to cap it) for how much of it a caller gets per request.

Next steps

- Turn your first SQL query into a REST API – the same `parameters` shape, from scratch.
- Connect to MongoDB – the `:name` placeholder idea applies to a Mongo filter document too, just substituted as a real JSON value instead of SQL text.

READ NEXT
Group related queries into a collection

QUESTIONS OR PROBLEMS
Discord · GitHub issues

LATEST VERSION OF THIS PAGE
anantharajuc.github.io/QueryAPIGate-website/Guide.dc.html?g=06-use-bound-parameters-safely

FOLLOW QUERYAPIGATE GitHub X Discord Bluesky Mastodon YouTube Reddit
@queryapigate on X, Bluesky, Mastodon and YouTube · r/queryapigate on Reddit · github.com/AnanthaRajuC/QueryAPIGate

QueryAPIGate · one governed gate between your databases and everything that asks them for data · self-hosted, Apache-2.0 · © 2026 QueryAPIGate